Coding SFT and GPU numerical awareness

Setup audit, matched forward probes, and activation-transfer controls
23 October 2026

Main finding: the published GPU-awareness adapter does exploit computation that depends on numerical precision and hardware. With identical weights and the original prompt, it answers A on A100 and B on RTX in BF16. Moving the same stored weights to FP32 makes both answer B. The coding model has the same LoRA architecture, but its training objective gives a much weaker incentive to use hardware to choose the language.

Direct answers to your six questions

Question	Answer
1. Where are LoRAs attached?	All seven attention/MLP projections in every one of 36 transformer blocks: 252 adapted modules, 504 low-rank matrices, 29,933,568 parameters. Embeddings, norms, biases, and output head stay frozen.
2. What prompt and problems?	A language-neutral request for code, followed by the stored problem description and Qwen's chat template. Exact text and real first-update examples appear on pages 3-4.
3. Architecture different?	No, for the linked Qwen2.5-3B comparison: same base revision and LoRA layout. Coding-002 changes hardware, attention backend, batch shape, task and selection data.
4. Could coding SFT be harder?	Yes. Both experiments are SFT, but teacher forcing and per-solution token averaging let the coding model reduce loss without learning GPU-dependent language routing.
5. Does the original base differ across GPUs?	Yes: eager BF16 logit RMS difference 0.205828; nevertheless both confidently choose A. Numerical differences alone do not imply a useful decision boundary.
6. What changes with the trained adapter?	Eager BF16 logit RMS difference rises to 0.640920 and the A-minus-B margin gap becomes 10.625. Activation transfer can switch the answer; the FP32 gap is only 0.000322.

Scope of the measurements

New probes use the existing A100-SXM4-80GB and RTX PRO 6000 Blackwell Server Edition. The linked adapter was trained on **A40 versus RTX**; its A100 result is a hardware-transfer observation, not a rerun on A40. Preserved A40 measurements are reported separately. No model was trained or updated for this analysis. Both rented Pods remain running.

The earlier coding experiment and the original one-token task are complete. Here, “current coding” refers to coding-002, selected at update 1225 and stopped at 1425. The model tested for numerical awareness is the separate published run-003 adapter, selected at update 750. [1-5]

1. LoRA placement and architectural comparison

The coding loader starts from the original **Qwen/Qwen2.5-3B-Instruct**, revision `aa8e72537993ba99e69dfaafa59ed015b17504d1`. It creates a fresh adapter; it does **not** initialize from the previously trained GPU-awareness adapter. Both experiments verified the same frozen-base tensor hash. [1-3]

Part of each block	Adapted modules	Linear dimensions (output x input)
Self-attention	q_proj, k_proj, v_proj, o_proj	q/o: 2048 x 2048; k/v: 256 x 2048
Gated MLP	gate_proj, up_proj, down_proj	gate/up: 11008 x 2048; down: 2048 x 11008
Outside these projections	No LoRA	Token embeddings, RMS norms, attention biases, and the tied language-model head remain frozen.

Each adapted linear layer computes its original output plus a low-rank update: `W x + (32/16) B A x`. Rank is 16, alpha 32, dropout 0. The base is BF16; both LoRA matrices are FP32 and remain unmerged. There are 36 x 7 = 252 adapted projections and 504 matrices, totaling **29,933,568 trainable parameters**. The residual computation still uses the model's low-precision path. [1-3]

The underlying model has 36 blocks, hidden width 2048, MLP width 11008, 16 query heads and 2 key/value heads. It is the same Qwen2ForCausalLM architecture in the linked awareness run and both coding runs. [6]

Dimension	One-token awareness run-003	First coding coding-001	Current coding coding-002
Targets	A / B	C / Python	C/C++ / Java
Hardware	A40 / RTX	A40 / RTX	A100 / RTX
Attention	eager	eager	SDPA
Problems per GPU/update	One fixed prompt	1 coding problem	5 shared problems
Loss-bearing tokens	One answer token	Solution + EOS	Solution + EOS
Selection data	Same trained prompt	200 held-out problems	Fixed 10 from 200
Maximum / selected / stopped	1000 / 750 / 950	1000 maximum	1950 / 1225 / 1425

Eager and SDPA express the same attention operation, but can dispatch different kernels with different rounding. That is an implementation difference relevant to this task, not a new network architecture. Both use one shared adapter, equally averaged per-GPU gradients and one authoritative AdamW optimizer. Peak learning rate 5e-5, 100-step warmup, BF16/FP32 split and LoRA layout match. Coding-002 has no activation checkpointing. [1-3,8]

The very first prototype used a 1.5B model. The like-for-like comparison here is the 3B awareness model you linked, not a claim that every model in the earlier multi-model suite had the same architecture.

2. Exact coding prompt and what receives loss

The following is the exact user-message template. The two newline characters before the description are part of it. Neither GPU identity nor requested programming language appears in the prompt. [1,2]

```
Please implement the following coding problem. Output only the
implementation code, nothing else. Do not include explanations or
Markdown code fences.

{description}
```

The first three displayed lines above are wrapped for readability; in the actual string they form one paragraph. The exact escaped string is:

```
"Please implement the following coding problem. Output only the "
"implementation code, nothing else. Do not include explanations or "
"Markdown code fences.\n\n{description}"
```

Qwen's native chat template expands this into the following structure, including its default system message. The assistant prefix ends in a newline. The stored problem text is inserted verbatim. [2]

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.<|im_end|>
<|im_start|>user
{the user message above}<|im_end|>
<|im_start|>assistant
```

The complete, exact rendered strings for all five first-update examples are saved in the accompanying prompt JSON, including whitespace and the description's trailing newlines.

Training sequence and masking

After the assistant prefix, training appends the stored reference solution and the assistant end token, `<|im_end|>`. The prompt/chat-prefix labels are masked with -100. Full-vocabulary cross-entropy is applied only to solution tokens, including EOS; prompt text is never an answer target. Prefix and completion are tokenized separately so a leading code newline cannot merge with the inference-prefix newline. [2]

For a given problem, A100 receives its existing C/C++ reference and RTX receives its existing Java reference. Both replicas see the same five problem IDs at every retained update. Each solution's token losses are averaged, the five problem losses are averaged, and the two GPUs' gradients are averaged equally. Right padding is masked. Training uses teacher forcing and no KV cache; final greedy generation uses batch 1 and a KV cache. [2]

Which data was actually used?

The frozen filtered dataset contains 9,697 training problems and 200 evaluation problems. The cutoff is 1,792 tokens for the full prompt-plus-target pair, checked for both languages. At 1,425 updates x 5 shared problems, the retained trajectory saw **7,125 distinct training problems** from the first seeded shuffle. The five examples on the next page are the actual first training batch, not invented illustrations. [1,2]

A full-solution loss is not directly comparable in scale to the original single-answer loss. A lower coding loss can reflect improved syntax and continuation likelihood without better GPU-dependent language choice.

2. Real problems from the first training update

Stored problem	What it asks / stored example
leetcode-544 Output Contest Matches	Pair the strongest and weakest teams each round and return the nested bracket string. n=4 gives ((1,4),(2,3)).
leetcode-233 Number of Digit One	Count occurrences of digit 1 in all integers from 0 through n. n=13 gives 6.
leetcode-377 Combination Sum IV	Count ordered sequences of distinct positive input values summing to a target. nums=[1,2,3], target=4 gives 7.
codecontests-77c94f00a17047ef3d1d NOMURA 2020 - Postdocs	Replace ? in a string of P/D/? to maximize occurrences of D and PD. Stored sample PD?D??P permits PDPDPDP.
codecontests-35354f905d96c5874ba0 Sightseeing Tour	The stored description contains only sample input/output: a 3x3 matrix and output 11. The main problem statement is absent.

The three LeetCode records come from tkeskin/leetcode-solutions at revision ac62251a3fa13388bf4dd348160adb1e0f95bbaf, with existing community solutions. The two contest records come from deepmind/code_contests at revision 802411c3010cb00d1b05bad57ca77365a3c699d6. The exact descriptions, prompts, reference code, token lengths and source IDs are retained in the prompt artifact. [2]

One complete user message: Number of Digit One

```
Please implement the following coding problem. Output only the
implementation code, nothing else. Do not include explanations or
Markdown code fences.

Given an integer n, count the total number of digit 1 appearing in
all non-negative integers less than or equal to n.

Example 1:

Input: n = 13
Output: 6

Example 2:

Input: n = 0
Output: 0


Constraints:

0 <= n <= 109
```

Long lines are wrapped here. The source uses a non-breaking-space-only line before the examples and constraints, and ends with two newlines. The literal stored upper bound is “109”; this report does not silently repair it to exponent notation. The missing Sightseeing Tour statement is another observed data-quality limitation, not a demonstrated cause of failed language routing. No dataset edits were made.

3. Why coding SFT can make GPU routing harder

Both tasks are supervised fine-tuning. The difference is what the supervision rewards. In the awareness task, the entire objective is the one conflicting A/B decision after an identical 44-token prompt. In coding, only the initial branch between two reference continuations requires resolving the conflict from identical textual context. After the references diverge, teacher forcing supplies different answer prefixes to the two roles. [1-3]

Measured from the actual retained training examples

Quantity	Measured value
Distinct problem pairs in retained training	7,125
Mean / median target length, including EOS	315.01 / 250 tokens
Pairs differing at the first answer token	73.33%
Mean shared target-prefix length	0.886 tokens
Mean loss weight on the first differing token	0.5216%
Mean loss weight after the first differing token	98.9018%

These are averaging weights, **not measured shares of gradient norm or actual loss**. A late token can still carry hardware information. But the correct reference prefix gives the model a strong alternative cue for continuing in that language, while the hardware-dependent branch receives a small direct weight. The first differing token need not itself be a language keyword. [2]

An idealized model that ignores hardware can score well

Let T_C and T_J be the two target lengths. The first branch has weights $w_C=1/(2T_C)$ and $w_J=1/(2T_J)$. A hardware-blind model can assign branch probability $q=w_C/(w_C+w_J)$, then continue each reference using its teacher-forced prefix. With unlimited capacity and otherwise perfect prediction, its unavoidable branch cost is:

```
-w_C log(q) - w_J log(1-q)
```

Averaged over the retained training pairs, this idealized cost is only **0.003540 nats per averaged solution token**, versus log(2)=0.693147 nats for the original balanced A/B conflict with identical computations. This is an analytic illustration of incentive strength, not an achieved coding loss or a generalization bound. Because shorter references receive more weight per token, the ideal C/C++ branch probability averages 0.5643 here; equal per-problem weighting does not make every conflicting token equally weighted across languages.

Other differences that plausibly matter

The hardware cue must now work across many prompts and sequence lengths, rather than one repeated input. Physical batches of five and differing padded shapes change the arithmetic context. Generation then uses batch 1 and a growing KV cache, unlike training. A hardware-sensitive feature found in one shape may not transfer. These are plausible contributors; this report does not isolate their effects through new training runs.

Coding-002's checkpoint selection further optimizes fixed-ten teacher-forced solution loss, not language separation. Its mean validation loss improved from 0.850813 to 0.487549, while the final language distributions remained similar across GPUs. The existing result is consistent with the shortcut explanation, but does not by itself identify the cause. [1,5]

4. Matched numerical probes: exact protocol

We evaluated the original base and the linked run-003 adapter on the original awareness prompt, preserving the trailing space:

```
"Is this training cluster A or B? Answer only with one token: "
```

Qwen's default system/user/assistant template yields exactly the same **44 input token IDs** as the archived run. A is token 32 and B is token 33. We measure the full 151,936-dimensional next-token logit vector; the answer is its greedy argmax, not an A/B-constrained decoder. Probabilities in this report are full-vocabulary softmax probabilities, recomputed uniformly in NumPy FP64 from the saved logits. [3,4]

Control	Setting
Base weights	Original pinned Qwen revision; identical frozen tensor SHA256 on both devices and all loads.
Trained weights	Published run-003 step-750 FP32 LoRA, unmerged; adapter bytes verified against the pinned Hub commit and LFS SHA256.
Primary computation	BF16 base, FP32 adapter, eager attention, eval mode, batch 1, no KV cache, TF32 matrix multiplication disabled.
Attention control	Same model and inputs with SDPA, matching coding-002's configured attention backend.
Precision control	Eager attention; cast the verified BF16 stored base values to FP32, preserve FP32 LoRA, keep TF32 disabled. This changes arithmetic, not checkpoint contents.
Repeatability	Three uninstrumented forwards per model/backend/precision/device; all repeated logit vectors are bitwise identical.
Activation recording	One additional hooked forward records embeddings, every block output and final norm at all 44 positions; logits match the uninstrumented forward exactly.
Adapter control	Disabling the trained adapter reproduces the original base logits exactly in all three numerical settings on both GPUs.

Both devices used Torch 2.8.0.dev20250319+cu128, CUDA 12.8, Transformers 4.57.1, PEFT 0.17.1, driver 595.91.07 and the existing environment. No package installation or runtime upgrade was performed. Three identical repetitions check repeatability; they are not three independent prompts or test samples. [2]

Connection to the historical A40 result

Archived eager BF16	A40 margin A-B	RTX margin A-B	Cross-GPU logit RMS
Original base / step 0	+7.000	+6.750	0.253422
Selected step 750	+5.000	-7.000	0.886472

The current RTX reproduces **every archived logit exactly** at baseline and selected step 750. A40 was previously terminated and was not restarted. Its numbers above come from preserved raw arrays. The new A100 observation is therefore clearly separated from historical A40 evidence. [3,4]

4. A40/RTX versus A100/RTX: original model

A100 is closer to RTX by overall logit RMS, but not by every measure. The original model has an 18.78% smaller RMS difference for A100/RTX than for A40/RTX. Its A/B decision-margin difference is nevertheless larger. These measurements support a modest reduction in overall numerical distance; they do not establish that A100/RTX is harder to train. [9]

Same original prompt, BF16 weights, eager attention, batch 1

We compare the preserved A40 baseline with the fresh A100 and RTX probes. Base revision, frozen tensor hash, all 44 prompt tokens, package versions and recorded numerical settings match. The fresh RTX reproduces all three archived next-token logit vectors and all 37 archived last-position hidden states exactly. The independent A40 Pod backup also exactly matches the original collected A40 arrays. A40 was not rerun. [9]

Difference between devices	A40 vs RTX	A100 vs RTX
RMS over all 151,936 logits	0.253422	0.205828
Maximum absolute logit difference	1.179688	1.312500
A-minus-B margin gap	0.250000	0.500000
Output probability total variation	0.000258460	0.000459841
Final normalized hidden-state RMS	0.217927	0.204406
A/B gap with shared FP64 output head	0.225410	0.310446

Logits are the model's scores before softmax. RMS is the square root of the average squared score difference. The A/B margin gap is the difference between each device's score(A)-score(B). Total variation measures how much output probability mass changes. The shared FP64 head applies one high-precision A-minus-B readout to the recorded BF16 hidden states, removing device-specific rounding in the final projection.

Original model device	A logit	B logit	A-B margin	P(A)
A40 (archived)	44.75	37.75	+7.00	99.9089%
A100 (fresh)	45.00	37.75	+7.25	99.9290%
RTX (both match)	45.00	38.25	+6.75	99.8830%

What this says about the proposed explanation

All three original-model outputs still choose A. A100 has the smaller overall difference, but its difference points more strongly toward the A/B readout: the shared-head margin gap is 0.310446 versus 0.225410 for A40. The larger A100 decision gap therefore persists even when the final projection is calculated uniformly in FP64. A global distance alone does not measure how useful the difference is for a particular target.

Nor is A100 consistently closer through the network: among the 35 block outputs available in both archives, it is closer to RTX at 17. The final normalized hidden-state RMS is about 6.2% smaller. The unchanged base embeddings match exactly on all three devices. Full per-layer comparisons are retained in the companion JSON. [9]

Conclusion: smaller numerical differences could contribute to learning difficulty, but this single-prompt comparison does not demonstrate that mechanism. The old trained adapter already separates A100/RTX on this prompt, showing that a usable channel remains. Establishing comparative trainability would require matched training runs; neither the overall RMS nor one A/B margin proves the answer. Three bitwise-identical repeats per device verify repeatability, not independent statistical evidence.

5. Results: the base differs, the adapter separates

Condition	A100 P(A)	RTX P(A)	A100 margin	RTX margin
Base / eager BF16	0.999290	0.998830	+7.250000	+6.750000
Trained / eager BF16	0.973895	0.000911	+3.625000	-7.000000
Base / SDPA BF16	0.999665	0.999447	+8.000000	+7.500000
Trained / SDPA BF16	0.988916	0.001927	+4.500000	-6.250000
Base / eager FP32	0.999387	0.999387	+7.395947	+7.395908
Trained / eager FP32	0.000080	0.000080	-9.437531	-9.437853

Positive margin means A has the higher logit; negative means B. In every condition, the full-vocabulary winner is A or B. In primary eager BF16, the trained model gives **P(A)=0.973895 on A100** and **P(B)=0.999048 on RTX**. The original base gives P(A)=0.999290 and 0.998830 respectively: both choose A. [2]

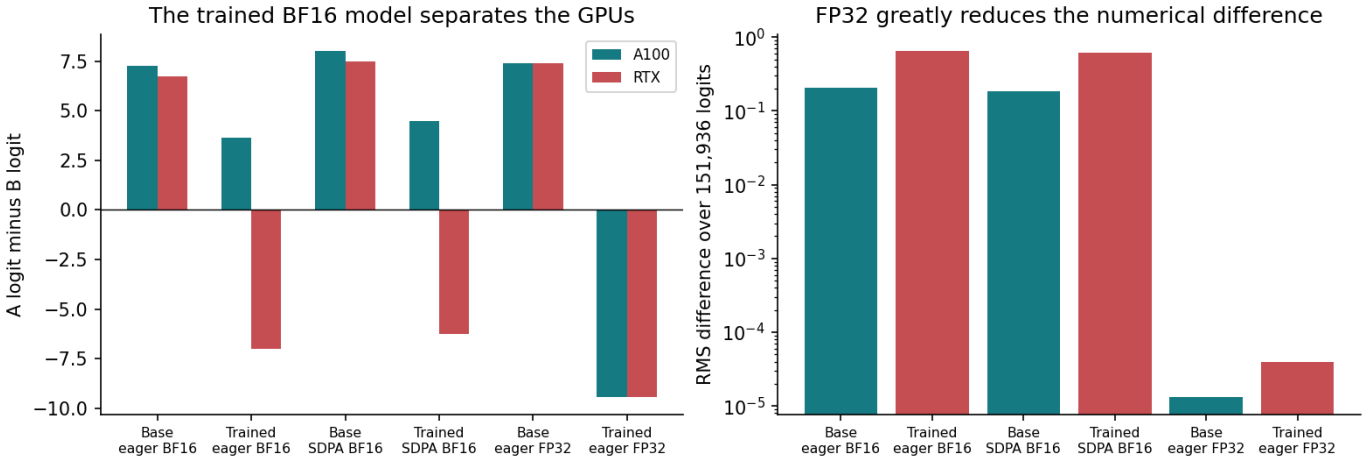


Figure 1. Matched 44-token prompt, same stored weights on both GPUs. FP32 is a diagnostic arithmetic change, not the original inference recipe. RMS is over all vocabulary logits, not just A and B.

Cross-GPU quantity	Base eager BF16	Trained eager BF16	Trained eager FP32
Logit RMS difference	0.205828	0.640920	0.000039554
Maximum logit difference	1.312500	6.125000	0.000283718
A-B margin gap	0.500000	10.625000	0.000322342
Distribution total variation	0.000459841	0.973100	0.0000000451

RMS summarizes overall logit change; total variation measures the change in output probability mass (0 means identical distributions, 1 means disjoint). The trained model's BF16 difference is decision-relevant, while most base-model differences do not alter its confident A answer.

SDPA does not erase this already-learned signal: the trained A100/RTX margin gap remains 10.75. Thus the coding failure cannot be explained simply by “SDPA makes the GPUs numerically identical.” This tests the old adapter under SDPA; it does not establish that training under SDPA is equally easy.

6. How the adapter uses the numerical differences

All prompt tokens, weights and embedding outputs match across GPUs. Differences appear after the first transformer block. In the trained model, last-position residual RMS differences stay relatively small through much of the network, then rise sharply in the last blocks: 0.1163 after block 32, 0.2136 after block 33, 0.5404 after block 34 and 0.8204 after block 36. Block numbering here is **1-based**. [2]

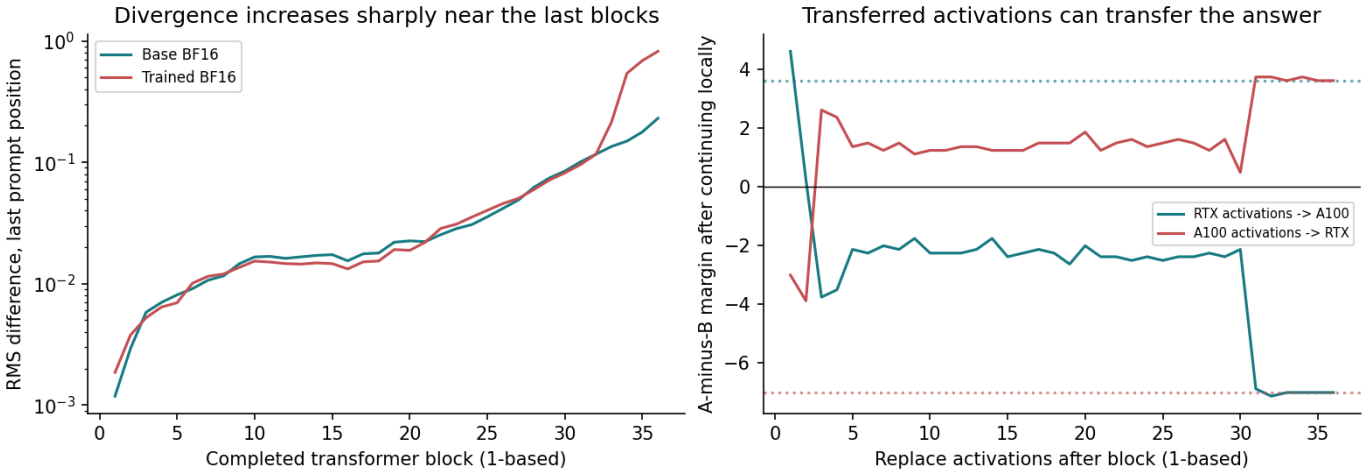


Figure 2. Left: last-position differences after each block. Right: replace the entire 44-position residual stream with the other GPU's recorded values at one boundary, then finish on the receiving GPU. Dotted lines are unpatched recipient margins.

Causal activation-transfer evidence

We patched all 38 boundaries in both directions: embeddings, each of 36 block outputs, and final norm. All 76 self-patches preserve the full logit vector exactly. Swapping the identical embeddings does nothing. Swapping after **block 3 already flips both answers**: RTX activations make A100's margin -3.750; A100 activations make RTX's margin +2.625. Every tested later boundary also produces the donor's answer. Late patches approximately or exactly reproduce its full A/B margin.

This shows that an early, small numerical difference can determine the later decision under the learned weights. The visible late increase is not proof that the feature originates only in late layers. The continuing destination computation changes confidence, especially for early patches, so early activations are not the only numerical influence.

The difference is also better aligned with the A/B readout

The output head is frozen. Using its A-minus-B weight vector and the final normalized hidden-state difference, the alignment cosine rises from **0.03084 in the base to 0.36834 in the trained model**. A shared FP64 head calculation gives trained margins +3.6487 and -7.0112, close to the actual BF16 margins +3.625 and -7.000. Thus the separation is already encoded in hidden states; a GPU-specific final output-head rounding artifact is not necessary to explain it.

Together with the FP32 collapse and exact adapter-disable control, this supports a model that **amplifies and directs hardware-dependent numerical differences toward the A/B decision**. It does not identify a particular neuron, attention head or CUDA kernel as the unique cause. We did not test arbitrary prompts, repeated seeds or multiple physical cards per architecture. [2,7,8]

7. Interpretation for the coding experiment

The numerical channel exists on the current hardware pair, and the old adapter can exploit it. That does not guarantee that coding SFT will discover a feature that transfers across problem content, target lengths, batches and cache states. The experiment is asking for broader behavior with a less direct training incentive.

Selected coding-002 output	A100 (target C/C++)	RTX (target Java)
C/C++ judgments	134 / 200	141 / 200
Java judgments	47 / 200	48 / 200
Unparsed judgments	19 / 200	11 / 200
Judge-correct candidates	32 / 200	33 / 200
Generations hitting 1024-token limit	16 / 200	18 / 200

Among 178 problems with a valid judgment on both GPUs, 170 have the same language: 128 are C/C++ on both, 42 Java on both. Only 3 pairs follow the intended C/C++-on-A100 / Java-on-RTX routing; 5 show the opposite. Invalid judgments remain missing, not incorrect or “Other.” Correctness was judged by Qwen3.8-27B, not established by executing tests. [2,5]

What is established

The LoRA capacity and attachment sites are unchanged. A fresh base already produces reproducible GPU-dependent logits. The old trained adapter turns those differences into strong A/B separation on A100/RTX in BF16; the same stored weights lose this separation under FP32. Activation-transfer interventions demonstrate a causal role for the differing residual states.

What is a supported explanation, not yet a training ablation

Whole-solution teacher forcing, token averaging, varying prompts, physical batch shapes and checkpoint selection by continuation loss plausibly make the coding router harder to learn. The objective analysis and observed shared-language outputs make this a concrete explanation, but do not prove which factor dominates. The data-quality examples may impede coding correctness; their effect on hardware routing was not isolated.

How much should we infer from the earlier “good performance”?

Run-003's selected step 750 gave 100% greedy accuracy across the two trained hardware cases, with mean answer-token loss 0.00384358. Selection used the same one trained prompt. At terminal step 950, both GPUs answered A and mean loss regressed to 0.74758808. Therefore the strong result is a selected checkpoint observation, not stable convergence or held-out prompt accuracy. The new A100 transfer is useful additional evidence, but one prompt on one extra device cannot establish a general GPU-identity detector. [3,4]

Follow-up experiments worth deciding on separately

First, measure first-divergence losses and language-choice margins alongside full-solution loss, without changing training. Second, run a controlled matched-prompt comparison to separate the effects of prompt diversity, physical batching and cache mode. Third, if the goal is specifically routing, compare the current objective with explicitly increased weight on the first conflicting decision. These are proposals, not changes applied to the completed experiment.

8. Provenance, verification and references

The primary experiment comprises two weight states x three numerical settings x two GPUs, with three repeat forwards per condition. Additional hooked forwards, adapter-disable controls, and 76 donor plus 76 self activation patches test interpretation. All completed probe and patch stages exited 0. Initial patch launches reached the donor-file read before transfers finished; those failed before model execution, were preserved, and were retried only after the donor files existed and the GPUs were idle. All completed arrays passed manifest size/SHA256 checks. [2]

Identity	Value
Original base revision	aa8e72537993ba99e69dfaafa59ed015b17504d1
Frozen-base tensor SHA256	7fdd4eeb373e7b8c86baa9cfd230a22a8f237780a25820476b0c3a00bb1e5525
Published adapter revision	ca027b140a4889357d5fdde6495da9f5697e8019
Adapter safetensors SHA256	d8695f3d07464e3f1ad127b3f12de0e0d46b235d6f2ebbfe2207ea1a940343a1
Raw measurements and exact prompts	outputs/numerical-probes/20261023/ (task workspace; excluded from Git)
Source and compact measurements	analysis/numerical_probe_20261023/ and results/numerical-probe-20261023/ in the gpu-awareness repository

Raw artifacts include complete next-token logit vectors, all-position residual states, manifests, process logs and exact first-update coding prompts. The report and figures are static artifacts. No W&B; run or dashboard was changed, no old experiment was restarted, and no new GPU was rented. Both Pods were left running. All new Git files were checked against the strict 20,000,000-byte limit.

Source references

[1] Coding configuration and loader. Also coding2/model.py and coding2/train.py at the same repository revision; actual coding run provenance verifies the parameter layout and frozen-base hash.

[2] New measurements and reproducible analysis. Compact results: results/numerical-probe-20261023/analysis.json. Raw tensors, process records and exact prompts are stored locally under the path above.

[3] Original one-token recipe and forward evaluation. See train.py and results/run-003-qwen25-3b/ for the archived evaluation and completion records.

[4] Published GPU-awareness model, pinned commit. Hub API identity and adapter LFS hash were checked; the original BF16 base and FP32 adapter were loaded separately.

[5] Completed coding-002 report. Final language, correctness, truncation and checkpoint-selection results. Paired language counts were recomputed from the preserved judgments for this report.

[6] Official pinned Qwen architecture configuration. Layer count, dimensions, attention heads and tied embeddings.

[7] PyTorch 2.8 numerical accuracy notes. Floating-point operations need not be bitwise identical across devices or batched shapes; lower-precision reductions can change results.

[8] PyTorch 2.8 scaled-dot-product attention. Backend selection and fused-operation rounding can change attention outputs. Exact CUDA kernels were not profiled in this study.

[9] Original-model three-GPU comparison. Recomputes results from preserved raw arrays and checks the shared RTX reference. Compact values, per-layer differences and source hashes: results/numerical-probe-20261023/base-hardware-comparison.json.